2024 Workshop on Fault Detection and Tolerance in Cryptography (FDTC) | 979-8-3503-8036-1/24/$31.00 ©2024 IEEE | DOI: 10.1109/FDTC64268.2024.00013

A Single-Trace Fault Injection Attack on Hedged Module Lattice Digital Signature Algorithm (ML-DSA)

Sönke Jendral
KTH Royal Institute of Technology
Stockholm, Sweden
jendral@kth.se

John Preuß Mattsson
Ericsson Research
Stockholm, Sweden
john.mattsson@ericsson.com

Elena Dubrova
KTH Royal Institute of Technology
Stockholm, Sweden
dubrova@kth.se

***Abstract*—Module Lattice Digital Signature Algorithm (ML-DSA) is a post-quantum digital signature algorithm currently being standardised by the NIST. Devices making use of ML-DSA are expected to soon become generally available in various environments. It is thus important to assess the resistance of ML-DSA implementations to physical attacks. This paper presents a fault injection attack on hedged ML-DSA in ARM Cortex-M4. First, voltage glitching is performed to skip computation of a seed during the generation of the signature. We identified settings that allowed us to consistently skip the necessary function without crashing the device. After the fault injection, the secret key vector $\mathbf{s}_1$ is derived directly from the resulting faulty signature. The attack succeeds in recovering $\mathbf{s}_1$ from a single trace with a probability of around 53%. We also propose countermeasures against the presented attack.**

***Keywords*-Fault injection; ML-DSA; Dilithium; CRYSTALS-Dilithium; PQC; Digital signature; Key recovery attack**

I. Introduction

The Module Lattice Digital Signature Algorithm (ML-DSA) is a quantum-resistant, lattice-based digital signature scheme chosen by the National Institute of Standards and Technology (NIST) [1] as the future general-purpose digital signature algorithm. ML-DSA is designed to be strongly unforgeable under chosen message attacks (SUF-CMA) in both classical and quantum random oracle models [2]. SUF-CMA security means that an adversary with access to the public key and a signing oracle cannot generate a valid signature for a new message nor produce an alternative valid signature for a previously signed message. The security of ML-DSA relies on the presumed hardness of the Module Learning with Errors (M-LWE) and Module Short Integer Solution (M-SIS) problems.

Before NIST standardization, ML-DSA was known as CRYSTALS-Dilithium or simply Dilithium. During the final phase of the standardization project, NIST introduced a "hedged" mode that replaces the deterministic mode as the default. In the hedged mode, the private seed ρ' is derived from the secret key, the message, and a pseudorandom string. The deterministic mode is retained as an option, but the fully randomized mode has been removed. Several fault injection attacks on Dilithium, which allow for the extraction of secret key vectors, have been demonstrated in the past, including [3], [4], [5], [6], [7]. However, to the best of our knowledge, the differences between CRYSTALS-Dilithium and ML-DSA have not been assessed in this context.

In today's digitalised world, devices running cryptographic algorithms are increasingly physically accessible to attackers. Often, these devices operate in resource-constrained scenarios, limiting the available security features. Simultaneously, the adoption of ML-DSA is expected to proceed rapidly across governments and various industries. ML-DSA is the sole general-purpose signature algorithm approved for use in US national security systems beyond 2030 [8]. The Internet Engineering Task Force (IETF) is actively working on integrating ML-DSA into security protocols such as X.509, Transport Layer Security (TLS), Internet Protocol Security (IPsec), and Secure Shell (SSH). These protocols are extensively used on the Internet, in enterprise networks, and cellular networks. The 3rd Generation Partnership Project (3GPP) intends to introduce ML-DSA and other quantum-resistant algorithms to 5G, the fifth generation of cellular network, as soon as final standards are published [9]. Therefore, it is crucial to assess the vulnerability of ML-DSA implementations to physical attacks, such as fault injection, to provide implementers the opportunity to address potential security issues.

1) Contributions: In this paper, we present a fault injection attack on an implementation of Dilithium in hedged mode. The presented attack requires only a single faulty signature to recover the secret key vector $\mathbf{s}_1$. Previous approaches making use of fault injection have not been applicable to non-deterministic versions of Dilithium, or required a higher number of signatures and faults, or generated signatures that do not pass the verification and can thus be detected.

We demonstrate a practical attack on a modified version of the software implementation of CRYSTALS-Dilithium by Abdulrahman et al. [10] to which we added the hedged mode. First, a voltage fault injection using the crowbar technique of O'Flynn [11] is performed to skip absorption of data during the computation of the hash ρ'. We identified settings that consistently skip the necessary function without crashing the device or disrupting other steps of the signature

2995-0252/24/$31.00 ©2024 IEEE
DOI 10.1109/FDTC64268.2024.00013

generation. From there, the secret key vector $\mathbf{s}_1$ can be directly derived from the generated faulty signature. Our attack succeeds to recover $\mathbf{s}_1$ from a single attempt with a probability of 0.528 ± 0.031.

2) Organisation of the paper: The rest of this paper is organised as follows. Section II provides background information on the ML-DSA algorithm and voltage fault injection. Section III describes previous work. Section IV presents the adversary model and attack scenario. Section V describes the experimental setup. Section VI presents the fault attack. Section VII describes the secret key recovery method. Section VIII summarises the experimental results. Section IX discusses possible countermeasures against the attack. Section X concludes the paper.

II. Background

This section describes the notation used in the remainder of this work, the ML-DSA algorithm specification, and the voltage fault injection method.

A. Notation

Let $\mathbb{Z}_q$ be the ring of integers modulo q also denoted by $\mathbb{Z}/q\mathbb{Z}$, R_q be the ring of polynomials $\mathbb{Z}_q[X]/(X^n+1)$ and T_q be the ring $\mathbb{Z}_q^n$. Lower-case regular font letters denote elements in $\mathbb{Z}_q$ or R_q, bold lower-case letters denote vectors with coefficients in R_q, the hat symbol $\hat{}$ denotes elements in T_q and upper-case letters are used for matrices. Let w_i denote the ith coefficient of the polynomial $w = w_0 + w_1 X + \cdots + w_{255} X^{255}$ and $v[i]$ denote the ith entry of a vector $\mathbf{v}$. The infinity-norm is given by $||\cdot||_\infty$, the concatenation of bit/byte strings a and b is given by $a||b$. The Boolean evaluation of an expression is denoted as $[\![\cdot]\!]$. The multiplication in $\mathbb{Z}_q$ or R_q is denoted by $\cdot$ and the multiplication in T_m is denoted by $\circ$ and performed entry-wise. Assignment from the result of a function or sampling from a set are denoted by $\leftarrow$. The blank symbol $\perp$ is used to indicate lack of an output.

B. ML-DSA algorithm

ML-DSA is derived from the latest version of CRYSTALS-Dilithium [2], and differs in an increased length for parameter $\hat{c}$ in parameter sets ML-DSA-65 and ML-DSA-87, an increased length of parameter tr, and the introduction of a "hedged" pseudorandom sampling procedure for the private seed ρ' that replaces the deterministic sampling procedure as the default. A modified variant of the deterministic sampling procedure is, however, retained for ML-DSA, while the previously present fully-randomised sampling method has been removed entirely. This paper focuses on the specifics of ML-DSA and the presented attack is applicable to both the deterministic and hedged modes in ML-DSA, but not to the randomised nor deterministic modes in CRYSTALS-Dilithium, which do not perform the hash computation targeted in the attack.

An overview over the possible sets of parameters is given in Table I. Additional details and descriptions can be found in the specification [1]. This paper focuses on ML-DSA-44 (respective Dilithium-2), though other variations can be approached similarly.

Dilithium is considered secure in the (Quantum) Random Oracle model based on the assumed hardness of the Module Learning-with-Errors (MLWE) and Module Shortest-Integer-Solution problems [12], [13]. The scheme uses the Fiat-Shamir with Aborts approach [14] in which an identification scheme is transformed into a signature scheme and rejection sampling is applied to sample a mask that prevents the secret key from being revealed through the signature.

The main components of the Dilithium scheme are the key generation procedure, the signing procedure and the verification procedure.

Algorithm 1 ML-DSA.KeyGen()

Output: Public key pk, secret key sk

1: $\xi \leftarrow \{0,1\}^{256}$
2: $(\rho, \rho', K) \in \{0,1\}^{256} \times \{0,1\}^{512} \times \{0,1\}^{256} \leftarrow \mathrm{H}(\xi, 1024)$
3: $\hat{\mathbf{A}} \leftarrow \mathsf{ExpandA}(\rho)$
4: $(\mathbf{s}_1, \mathbf{s}_2) \leftarrow \mathsf{ExpandS}(\rho')$
5: $\mathbf{t} \leftarrow \mathsf{NTT}^{-1}(\hat{\mathbf{A}} \circ \mathsf{NTT}(\mathbf{s}_1)) + \mathbf{s}_2$
6: $(\mathbf{t}_0, \mathbf{t}_1) \leftarrow \mathsf{Power2Round}(\mathbf{t}, d)$
7: $pk \leftarrow \mathsf{pkEncode}(\rho, \mathbf{t}_1)$
8: $tr \leftarrow \mathrm{H}(\mathsf{BytesToBits}(pk), 512)$
9: $sk \leftarrow \mathsf{skEncode}(\rho, K, tr, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0)$
10: **return** (pk, sk)

Figure 1. ML-DSA.KeyGen algorithm [2], [1].

1) Key generation (Algorithm 1): The key generation samples the matrix $\mathbf{A}$ and secret key polynomial vectors $\mathbf{s}_1$ and $\mathbf{s}_2$ by generating and expanding a random seed using SHAKE256. The coefficients of $\mathbf{s}_1$ and $\mathbf{s}_2$ are short, i.e. are in the range $[-\eta, \eta]$. It then computes $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$, which forms a part of the public key. Dilithium applies compression to $\mathbf{t}$ by dropping the d least significant bits to reduce its size, but an attacker is assumed to be able to recover the full value of $\mathbf{t}$. The most significant bits $\mathbf{t}_1$ and the seed ρ used for expanding the matrix $\mathbf{A}$ form the public key. The secret key includes the seed ρ, in addition to a private random seed K and the hash tr of the public key for use during signing, as well as vectors $\mathbf{s}_1$ and $\mathbf{s}_2$ and the d least significant bits $\mathbf{t}_0$.

2) Signing (Algorithm 2): The signing procedure computes the message representative μ by hashing the hash of the public key and the message using SHAKE256. It then computes an additional private random seed ρ' by hashing the private random seed K, a 256-bit random value rnd (or in deterministic mode, the value $\{0\}^{256}$) and the message representative μ. The seed ρ' is used to sample the nonce $\mathbf{y}$

Table I
ML-DSA PARAMETER SETS FROM [1].

Parameter set	n	q	d	τ	γ_1	γ_2	(k,l)	η	β	ω
ML-DSA-44	256	8380417	13	39	2^{17}	$(q-1)/88$	$(4,4)$	2	78	80
ML-DSA-65	256	8380417	13	49	2^{19}	$(q-1)/32$	$(6,5)$	4	196	55
ML-DSA-87	256	8380417	13	60	2^{19}	$(q-1)/32$	$(8,7)$	2	120	75

Algorithm 2 ML-DSA.Sign(sk, M)

Input: Secret key sk, message M
Output: Signature σ

$(\rho, K, tr, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0) \leftarrow \mathsf{skDecode}(sk)$
$\hat{\mathbf{s}}_1 \leftarrow \mathsf{NTT}(\mathbf{s}_1)$
$\hat{\mathbf{s}}_2 \leftarrow \mathsf{NTT}(\mathbf{s}_2)$
$\hat{\mathbf{t}}_0 \leftarrow \mathbf{NTT}(\mathbf{t}_0)$
$\hat{\mathbf{A}} \leftarrow \mathsf{ExpandA}(\rho)$
$\mu \leftarrow \mathrm{H}(tr||M, 512)$
$rnd \leftarrow \{0,1\}^{256}$ ▷ *Deterministic mode:* $rnd \leftarrow \{0\}^{256}$
$\rho' \leftarrow \mathrm{H}(K||rnd||\mu, 512)$
$\kappa \leftarrow 0$
$(\mathbf{z}, \mathbf{h}) \leftarrow \perp$
while $(\mathbf{z}, \mathbf{h}) = \perp$ **do**
  $\mathbf{y} \leftarrow \mathsf{ExpandMask}(\rho', \kappa)$
  $\mathbf{w} \leftarrow \mathsf{NTT}^{-1}(\hat{\mathbf{A}} \circ \mathsf{NTT}(\mathbf{y}))$
  $\mathbf{w}_1 \leftarrow \mathsf{HighBits}(\mathbf{w})$
  $\tilde{c} \in \{0,1\}^{2\lambda} \leftarrow \mathrm{H}(\mu||\mathsf{w1Encode}(\mathbf{w}_1), 2\lambda)$
  $(\tilde{c}_1, \tilde{c}_2) \in \{0,1\}^{256} \times \{0,1\}^{2\lambda-256} \leftarrow \tilde{c}$
  $c \leftarrow \mathsf{SampleInBall}(\tilde{c}_1)$
  $\hat{c} \leftarrow \mathsf{NTT}(c)$
  $cs_1 \leftarrow \mathsf{NTT}^{-1}(\hat{c} \circ \hat{\mathbf{s}}_1)$
  $cs_2 \leftarrow \mathsf{NTT}^{-1}(\hat{c} \circ \hat{\mathbf{s}}_2)$
  $\mathbf{z} \leftarrow \mathbf{y} + cs_1$
  $\mathbf{r}_0 \leftarrow \mathsf{LowBits}(\mathbf{w} - cs_2)$
  if $||\mathbf{z}||_\infty \geq \gamma_1 - \beta$ **or** $||\mathbf{r}_0||_\infty \geq \gamma_2 - \beta$ **then**
    $(\mathbf{z}, \mathbf{h}) \leftarrow \perp$
  else
    $c\mathbf{t}_0 \leftarrow \mathsf{NTT}^{-1}(\hat{c} \circ \hat{\mathbf{t}}_0)$
    $\mathbf{h} \leftarrow \mathsf{MakeHint}(-c\mathbf{t}_0, \mathbf{w} - c\mathbf{s}_2 + c\mathbf{t}_0)$
    if $||c\mathbf{t}_0||_\infty \geq \gamma_2$ **or** # of 1's in $\mathbf{h} > \omega$ **then**
      $(\mathbf{z}, \mathbf{h}) \leftarrow \perp$
  $\kappa \leftarrow \kappa + l$
$\sigma \leftarrow \mathsf{sigEncode}(\tilde{c}, \mathbf{z} \bmod^{\pm} q, \mathbf{h})$
return σ

Figure 2. ML-DSA.Sign algorithm [1].

from which the signer commitment $\mathbf{w}_1$ is computed as the high bits of $\mathbf{w} = \mathbf{A}\mathbf{y}$. The commitment hash $\tilde{c}$ is derived from $\mathbf{w}_1$ and μ and used to sample the challenge c. The signer's response is calculated as $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$ and its validity is checked to restart if necessary (i.e. the signing is aborted as per the Fiat-Shamir with Aborts approach [14]). Finally, the hint $\mathbf{h}$ is computed, which allows a verifier to reconstruct the entire value of $\mathbf{w}_1$. The values $\tilde{c}, \mathbf{z}$ and $\mathbf{h}$ form the signature σ.

Algorithm 3 ML-DSA.Verify(pk, M, σ)

Input: Public key pk, message M, signature σ
Output: Boolean

$(\rho, \mathbf{t}_1) \leftarrow \mathsf{pkDecode}(pk)$
$(\tilde{c}, \mathbf{z}, \mathbf{h}) \leftarrow \mathsf{sigDecode}(\sigma)$
if $\mathbf{h} = \perp$ **then return** false
$\hat{\mathbf{A}} \leftarrow \mathsf{ExpandA}(\rho)$
$tr \leftarrow \mathrm{H}(\mathsf{BytesToBits}(pk), 512)$
$\mu \leftarrow \mathrm{H}(tr||M, 512)$
$(\tilde{c}_1, \tilde{c}_2 \in \{0,1\}^{256} \times \{0,1\}^{2\lambda-256} \leftarrow \tilde{c}$
$c \leftarrow \mathsf{SampleInBall}(\tilde{c}_1)$
$\mathbf{w}'_{\mathrm{Approx}} \leftarrow \mathsf{NTT}(\hat{\mathbf{A}} \circ \mathsf{NTT}(\mathbf{z}) - \mathsf{NTT}(c) \circ \mathsf{NTT}(\mathbf{t}_1 \cdot 2^d))$
$\mathbf{w}'_1 \leftarrow \mathsf{UseHint}(\mathbf{h}, \mathbf{w}'_{\mathrm{Approx}})$
$\tilde{c}' \leftarrow \mathrm{H}(\mu||\mathsf{w1Encode}(\mathbf{w}'_1), 2\lambda)$
return $[\![||\mathbf{z}||_\infty < \gamma_1 - \beta]\!]$ **and** $[\![\tilde{c} = \tilde{c}']\!]$ **and** $[\![$ # of 1's in $\mathbf{h} \leq \omega]\!]$

Figure 3. ML-DSA.Verify algorithm [2], [1].

3) Verification (Algorithm 3): The verification procedure derives the challenge c from the signature and computes $\mathbf{w}'_{\mathrm{Approx}} = \mathbf{A}\mathbf{y} - c\mathbf{t}_1 \cdot 2^d$. Using the hint $\mathbf{h}$, the value $\mathbf{w}'_1$ is reconstructed from $\mathbf{w}'_{\mathrm{Approx}}$ and the commitment hash $\tilde{c}'$ is derived. The verification passes if the commitment hashes match and the additional validity criteria are fulfilled.

C. Fault injection techniques

Among the different techniques for fault injection, four main approaches for performing low-cost, minimally invasive fault injection can be identified: software-based glitching, Electromagnetic (EM) glitching, clock glitching and voltage glitching [15], [11].

1) Software-based glitching: This technique exploits properties of the executing hardware available through software. The various approaches differ from each other substantially, but include, for example, use of the Dynamic Voltage and Frequency Scaling feature to induce byte-corruption faults through overclocking, as in the CLKscrew attack [16], or bit flips caused by repeated row access in a DRAM chip, as in the RowHammer attacks [17].

2) EM glitching: This technique involves precise placement of a probe from which an EM pulse is emitted. This pulse can affect clock signals or inject additional power into the chip [18]. Several fault injection attacks on Dilithium [5], [19], [20] employ EM glitching, making use of both its ability to skip instructions by affecting the clock and its ability to flip/zeroise bits in memory.

3) Clock glitching: This technique exploits manipulation of the clock signal by injecting or withholding rising edges, thus altering the execution of instructions [18], [11]. Clock glitching has been employed in several attacks on Dilithium [7], [21], [6], [4], which make use of its ability to skip instructions.

4) Voltage fault injection: This technique uses manipulation of the power which is supplied to a processor to cause faults. Some approaches [22], [23] use uniform underpowering to slow down logic gates to cause faults. These approaches do not require precise timing control, but also offer limited control over affected instructions and the resulting fault. Other approaches [11], [24] instead use a precisely timed spike in the voltage to cause faults. While these approaches thus require greater timing control, they also allow specific instructions to be affected.

In this paper, we use the voltage fault injection technique of O'Flynn [11]. This technique uses a crowbar circuit to short the power rails of the processor, which induces oscillations in the target circuit, thus causing faults.

5) Fault model: Using the previously described voltage fault injection technique, the resulting faults are expected to include both (single/multiple) instruction skipping and instruction corruption. While the presented attack does not make use of instruction corruption, we empirically observed instances of corrupted instructions mentioned here for completeness.

III. Previous work

This section describes previous side-channel and fault injection attacks on cryptographic algorithms related to the presented work.

Bindel et al. [25] presented a number of fault injection attacks against lattice-based signature schemes. One of their approaches is a randomisation attack changing individual coefficients of $\mathbf{s}_1$, allowing for the recovery from multiple signatures. They also propose skipping the addition during the computation of $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$, thus allowing for the recovery of $\mathbf{s}_1$ from the same signature. However, their attacks do not target Dilithium and have not been experimentally verified against it. It therefore remains unclear how applicable their approaches are in practice.

In a subsequent work, Ravi et al. [3] demonstrated that skipping the entire addition of $\mathbf{y}$ is not necessary. Instead, they present a fault injection attack targeting the addition of single coefficients. This allows for recovery of the full vector $\mathbf{s}_1$ in around 1000–2000 faulty signatures (corresponding to the same number of traces, as they report a fault probability of 100%).

Recently, Krahmer et al. [4] extended this addition-skipping attack to randomised Dilithium and additionally presented an attack targeting the matrix $\hat{\mathbf{A}}$. As both approaches perturb single coefficients in the computation of $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$ in the signing procedure, they are able to use the verification to exhaustively search and correct the perturbation, thus allowing for recovery of a single coefficient. They practically demonstrate recovery of the full vector $\mathbf{s}_1$ using clock glitching in 22,952 traces.

Separately, Ravi et al. [5] proposed a fault attack exploiting zeroisation of twiddle constants in the Number Theoretic Transform (NTT). They presented two different attacks targeting deterministic and randomised Dilithium. The first zeroises the NTT of c, thus allowing for recovery of $\mathbf{y}$, which can be used to extract $\mathbf{s}_1$ from a different signature. The second attack requires computation of the signature in the NTT domain and zeroises most of the coefficients of $\mathbf{y}$, directly allowing recovery from the signature. They report recovery of the full secret key vector $\mathbf{s}_1$ from 13 and 3 faulty signatures respectively. While they do not report the number of traces required to gather the necessary signatures, they report a fault probability of 26% and 51% respectively for the attacks.

Espitau et al. [26] proposed a fault injection attack targeting the generation of $\mathbf{y}$ in a number of Fiat-Shamir with Aborts-based signature schemes. Their approach works by aborting the loop that performs the sampling of $\mathbf{y}$, thus leaving a number of coefficients uninitialised. A signature generated from such a faulty vector will likely directly contain several of the coefficients of the product $c\mathbf{s}_1$, allowing for the recovery of the secret key vector. This approach was extended by Ulitzsch et al. [6] to an implementation of Dilithium with fault countermeasures. They show that using an Integer Linear Program, they are able to recover the full secret key vector $\mathbf{s}_1$ using five faulty signatures gathered from 53 traces using clock glitching.

Bruinderink et al. [7] demonstrated a differential fault attack against the deterministic version of Dilithium. After generating a signature $\mathbf{z}$ for a message, they use fault injection to induce nonce-reuse to obtain a second signature $\mathbf{z}'$ computed for the same $\mathbf{y}$, but a different c. The secret key vector can then be recovered from the difference of the signatures $\mathbf{z} - \mathbf{z}'$. While they do not report the number of traces necessary for exploitation, their attack works with

faults anywhere in a large range of the execution, thus making it very likely the faulty signature could be obtained from a single attempt, thus allowing for recovery of the full key $\mathbf{s}_1$ from only two signatures.

Bauer and De Santis [27] showed a single-fault attack targeting the verification procedure of Dilithium (and another NIST PQC candidate Falcon). The attack works by first submitting a specifically crafted signature for a message and then skipping the subtraction of $c\mathbf{t}_1$ during the verification procedure using fault injection, thus causing the device to accept the signature as valid. The attack only requires a single fault to skip the subtraction, though they do not conduct experimental evaluation of their attack nor report the success rate for the fault.

There have also been single-trace side-channel attacks on Dilithium that target recovery of the secret key. The first of these is the work by Wang et al. [28] who demonstrated recovery of the secret key vector $\mathbf{s}_1$ through a profiled side-channel attack. They exploit leakage of the coefficients of secret key vectors $\mathbf{s}_1$ and $\mathbf{s}_2$ during the unpacking of the secret key at the first step of the signing procedure, thus their attack is also applicable to implementations using the hedged mode. By recovering half of these coefficients, they are able to solve a system of linear equations that enables recovery of the full vector $\mathbf{s}_1$ with a probability of 9% from a single trace.

The second is the work by Qiao et al. [29], who introduced a method for recovering the secret key vector $\mathbf{s}_1$ from the polynomial addition $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$, which is also applicable to implementations using the hedged mode. They employ a linear regression-based profiled attack to recover the value $\mathbf{y}$ from the polynomial unpacking procedure and a CNN-based attack to recover the value $c\mathbf{s}_1$ from the Montgomery reduction. By combining the leaked values, they are able to utilise Integer Linear Programming to recover the secret key vector $\mathbf{s}_1$ with a probability of 20% from a single trace.

There have also been single-trace side-channel attacks that aim to recover the input to Keccak-based hash functions. Kannwischer et al. [30] presented a soft-analytical side-channel attack that combines information gained from template matching in a factor graph. By applying belief propagation to the graph, they are able to recover 128- and 256-bit secret inputs from a single simulated trace with perfect probability 100% on 8-bit and 16-bit devices and slightly lower probability on 32-bit devices under realistic noise levels. They also propose masking and hiding (specifically shuffling) as countermeasures against their attack.

You and Kuhn [31] introduced a different template attack that instead uses enumeration of likely byte values in the state by combining multiple likelihood tables. Using their approach, they are able to recover 576-bit secret inputs of SHA3-512 in a single trace with a probability of 99% on a 8-bit device. Later, You and Kuhn [32] combined ideas from both approaches, which allowed them to recover 1088 bits of secret input to SHAKE256 from a single trace with a probability of 99.9% on a 32-bit device.

IV. Adversary model and attack scenario

This section defines assumptions about the adversary, their capabilities and goals in accordance with the adversary model in [33].

1) Assumptions: It is assumed that the adversary has physical access to the device under attack and access to equipment that allows for fault injection to be performed during the execution of the signing procedure. It is further assumed that the adversary is an outsider without access to privileged information that has a detailed understanding of the implementation of Dilithium run on the device under attack (e.g. from reverse-engineering).

2) Capabilities: The adversary is capable of detecting the execution of the signing procedure on the device under attack and injecting a precise voltage fault during the execution of the procedure, as well as observing its output. Note that the adversary does not need to control the message being signed, only detecting the signing and observing its output are necessary.

3) Goals: The goal of the adversary is to perform an existential forgery, i.e. to generate a signature σ for an adversary-chosen message M that passes verification using the public key of the device under attack. Is is known that knowledge of secret key vector $\mathbf{s}_1$ is sufficient to achieve an existential forgery [3].

A. Attack scenario

The attacker triggers or detects execution of the ML-DSA.Sign procedure on the device under attack. During the computation of the private random seed ρ', a fault is injected to fix its value to a known constant. The attacker then acquires the generated signature σ and derives the secret key vector $\mathbf{s}_1$ from it. Note that fixing the value of ρ' does not cause the generated signature to be invalid, thus circumventing any countermeasure that checks the validity of the signature after signing, as proposed by [25], [21].

V. Experimental Setup

This section describes the equipment used for the fault injection, as well as the target software implementation of ML-DSA.

A. Equipment

For the experiments, a ChipWhisperer-Husky, a CW313 adapter board and a CW308T-STM32F4 target device are used (see Figure 4).

The target device contains an ARM Cortex-M4-based STM32F415RGT6, which runs at a frequency of 16 MHz.

To avoid having to make any changes to the source code to trigger the fault injection, ARM CoreSight ETM/DWT watchpoints are used. In a real attack scenario, alternative

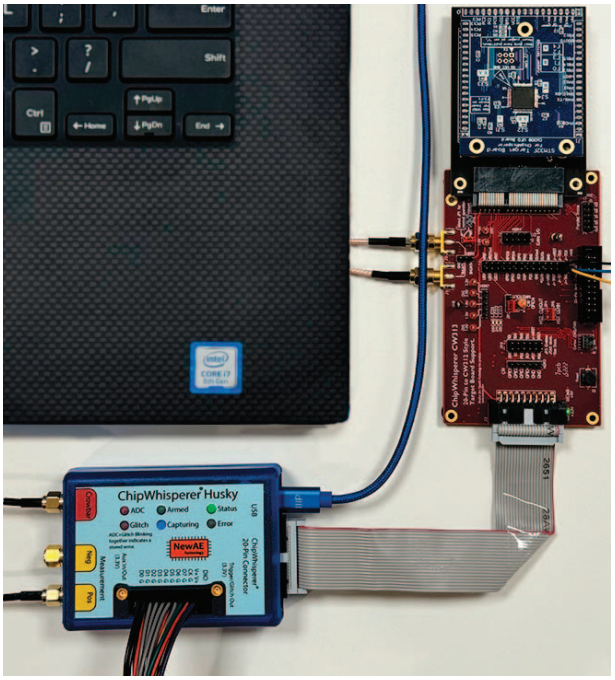

Figure 4. ChipWhisperer-Husky, CW313 adapter board and CW308T-STM32F4 board used in the experiments.

trigger sources such as reference waveforms of the power consumption or communication of the processor with peripheral devices can be used.

B. Target implementation

```
1  /* Compute message representative μ ... */
2
3  #ifdef DILITHIUM_USE_HEDGED_MODE
4    /* Hedged rnd ← {0,1}^256 */
5    randombytes(rnd, SEEDBYTES);
6  #else
7    /* Deterministic rnd ← {0}^256 */
8    memset(rnd, 0, SEEDBYTES);
9  #endif
10   /* Compute ρ' ← H(K||rnd||μ,512) */
11   shake256(rhoprime, CRHBYTES, key,
12            2*SEEDBYTES + CRHBYTES);
13
14 /* Expand matrix A ... */
```

Figure 5. The modified C code of the signing procedure of the CRYSTALS-Dilithium implementation of [10] with hedged mode added. The SHAKE256 function containing the target for the fault injection is highlighted in green.

In the experiments, a modified version of the CRYSTALS-Dilithium implementation by Abdulrahman et al. [10] is used, in which we added the hedged mode for sampling ρ' to the `crypto_sign_signature` procedure, as shown in Listing 5. We believe this implementation to be representative for other implementations of the hedged mode, as it follows directly from the specification. Note that the presented attack targets the implementation of the `shake256` procedure, which has not been modified.

The implementation is compiled using `arm-none-eabi-gcc` with the highest optimization level `-O3` (recommended default).

VI. Fault Injection Attack

This section describes the fault injection attack method and the implementation of the SHAKE256 algorithm.

A. SHAKE256 algorithm

SHAKE256 is an extendable output function based on the Keccak family of permutations [34]. Keccak employs a so-called sponge construction [35] alongside an iterated permutation function KECCAK-f to realise a function with arbitrary output length. In a sponge construction, input data is first absorbed into a state, after which the state can be squeezed to generate the output of the function.

In the implementation of [10], the SHAKE256 algorithm is implemented using four high-level functions. The `keccak_inc_init` function zero-initialises the Keccak state (represented as an array of 200 bytes). The `keccak_inc_absorb` function (see Listing 6) absorbs an arbitrary number of input bytes into the sponge. The `keccak_inc_finalize` function finalises the absorption of data and prepares for the extraction of output by applying a padding. Finally, the `keccak_inc_squeeze` function extracts an arbitrary number of output bytes from the state.

To expand an input value into an output value, these functions are called in the order they are listed here. In case the input values are not stored sequentially in memory, the `keccak_inc_absorb` function may be called multiple times to read from the different input buffers, though this is not the case for the sampling of ρ' in the implementation in Listing 5.

B. Main idea

The attack targets the absorption of data into the sponge during the hash calculation of ρ'. Specifically, a single voltage fault is used to skip the branching to the `KeccakF1600_StateXORBytes` function (see line 11 of Listing 6 and line 5 of Listing 7). Note that the loop in lines 3 to 9 is never executed, because the message length of 64 bytes for the message $K||rnd||\mu$ is less than the 136 bytes required to trigger a permutation. As such, skipping the `KeccakF1600_StateXORBytes` function is sufficient for the sponge to be left empty, see Figure 8. This allows an attacker to predict the output ρ' of the hashing procedure.

```
size_t keccak_inc_absorb(uint64_t *state, size_t bytes_not_permuted,
                         uint8_t *m, size_t mlen) {
  while (mlen + bytes_not_permuted >= 136) {
    KeccakF1600_StateXORBytes(state, m, bytes_not_permuted);
    mlen -= 136 - bytes_not_permuted;
    m += 136 - bytes_not_permuted;
    bytes_not_permuted = 0;
    KeccakF1600_StatePermute(state);
  }

  KeccakF1600_StateXORBytes(state, m, bytes_not_permuted, mlen);
  return bytes_not_permuted + mlen;
}
```

Figure 6. The C code of the `keccak_inc_absorb` procedure. The function targeted by the fault injection is highlighted in green.

```
...
mov r1, r8
mov r3, r4
mov r0, r7
bl KeccakF1600_StateXORBytes
ldr r2, [sp, #208]
ldr r3, [sp, #212]
...
```

Figure 7. Excerpt of the assembly code of the `keccak_inc_absorb` procedure. The branch targeted by the fault injection is highlighted in green.

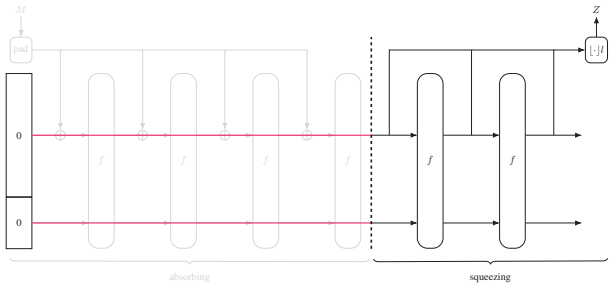

Figure 8. SHAKE256 sponge construction (adapted from [35]). The fault injection skips the absorption step, as highlighted in red. The output Z is then constant and independent of input M.

VII. Secret key recovery

This section describes the method used to recover the secret key vector $\mathbf{s}_1$ from a generated faulty signature.

Recovering the secret key vector $\mathbf{s}_1$ from a signature for a known value of ρ' is straightforward. A potential approach is shown in Algorithm 9. It works by reconstructing the commitment hash $\tilde{c}'$ using ρ' and a guess for the value κ. If the commitment hashes match, the challenge c is reconstructed and the secret key vector $\mathbf{s}_1$ can be computed as $\mathbf{s}_1 = (\mathbf{z} - \mathbf{y}) \cdot c^{-1}$, where c^{-1} is the inverse of c in T_q.

Note that it is possible for c to have entries with value 0 in the NTT domain. Those entries are not invertible in T_q and the corresponding entries of $\mathbf{s}_1$ in NTT domain thus cannot be determined. Empirically, we found this to rarely be the case (sampling 1M random challenges c, we found 21 tuples, all of which contained exactly one entry with value 0). Indeed, we may postulate that the probability for any one coefficient of c to be 0 in the NTT domain is $\frac{1}{q}$. However, despite the NTT maintaining uniformly random distributions throughout the mapping, the value of c is strictly speaking not uniformly and randomly distributed, as its Hamming weight is bounded by τ, so this claim may not hold precisely. Nonetheless, we propose to simply enumerate the possible values of $\mathbf{s}_1$ in NTT domain (i.e. enumerate all possible values of the entry in $\mathbb{Z}_q$) when encountering this case. For the sake of simplicity, this enumeration procedure is omitted from Algorithm 9. Note further that the choice of parameters for Dilithium is such that the expected number of iterations in the enumeration of κ is low (around 4) [2], thus this approach is generally efficient.

VIII. Experimental Results

This section describes the results of the fault injection attack and subsequent secret key recovery.

A. Glitch settings

We identified settings that consistently skip the desired function without crashing the device or disrupting other steps of the signature generation by conducting a grid search over the set of parameters offered by the ChipWhisperer-Husky. The results here were achieved by using the `'enable_only'` mode to insert a glitch lasting five clock cycles using both the high-power and low-power crowbar MOSFETs at an offset of 700 units.[1] Using these settings, we managed to successfully skip execution of the `KeccakF1600_StateXORBytes` function in 528 of 1000 attempts. This gives an estimate (95% confidence

[1]These units are dimensionless and depend on the internal frequency of the ChipWhisperer-Husky, but the offset corresponds to the distance between the rising edge of the clock cycle and the beginning of the glitch.

Algorithm 4 RecoverSecretKey($pk, \sigma, \rho', \kappa_{\max}$)

Input: Public key pk, signature σ, seed ρ', bound $\kappa_{\max}$
Output: Secret key polynomial $\mathbf{s}_1$

$(\rho, \mathbf{t}_1) \leftarrow \mathsf{pkDecode}(pk)$
$\hat{\mathbf{A}} \leftarrow \mathsf{ExpandA}(\rho)$
$(\tilde{c}, \mathbf{z}, \mathbf{h}) \leftarrow \mathsf{sigDecode}(\sigma)$
$\kappa \leftarrow 0$
while $\kappa < \kappa_{\max}$ **do**
  $\mathbf{y} \leftarrow \mathsf{ExpandMask}(\rho', \kappa)$
  $\mathbf{w} \leftarrow \mathsf{NTT}^{-1}(\hat{\mathbf{A}} \circ \mathsf{NTT}(\mathbf{y}))$
  $\mathbf{w}_1 \leftarrow \mathsf{HighBits}(\mathbf{w})$
  $\tilde{c}' \in \{0,1\}^{2\lambda} \leftarrow \mathrm{H}(\mu || \mathsf{w1Encode}(\mathbf{w}_1), 2\lambda)$
  if $[\![\tilde{c} = \tilde{c}']\!]$ **then**
    $(\tilde{c}_1, \tilde{c}_2) \in \{0,1\}^{256} \times \{0,1\}^{2\lambda - 256} \leftarrow \tilde{c}$
    $c \leftarrow \mathsf{SampleInBall}(\tilde{c}_1)$
    $\hat{c} \leftarrow \mathsf{NTT}(c)$
    $\mathbf{s}_1 \leftarrow \mathsf{NTT}^{-1}((\mathsf{NTT}(\mathbf{z}) - \mathsf{NTT}(\mathbf{y})) \circ \hat{c}^{-1})$
    return $\mathbf{s}_1$
  $\kappa \leftarrow \kappa + l$
return $\perp$

Figure 9. RecoverSecretKey algorithm.

interval) of the success probability of 0.528 ± 0.031. We believe that it is possible to further increase the success rate of the fault injection through additional optimisation of parameters.

B. Secret key recovery

We applied Algorithm 9 on the signatures generated during the first phase of the attack. As a guess for ρ', we use the output of SHAKE256 generated by applying the finalisation step on an empty state, which is a constant value easily derived by an attacker. Note that Algorithm 9 handles cases in which the fault injection was unsuccessful by limiting the number of iterations using the bound $\kappa_{\max}$, thus no additional processing of the signatures is required. We managed to successfully recover the secret key vector $\mathbf{s}_1$ for all 52.8% of cases where the fault injection was successful. In all of these signatures, the coefficients of c were nonzero and thus no additional enumeration was required.

IX. Countermeasures

The presented attack would be infeasible if the individual steps of the Keccak algorithm were inlined into the SHAKE256 routine instead of being separated into multiple subroutines. In fact, because the length of the parameters used in the calculation of ρ' is fixed, it would even be possible to eliminate control flow operations entirely, though such an implementation may not be practical.

Implementations of the SHAKE256 routine should also verify that after absorbing data into the sponge and before squeezing output from the sponge, the state is not empty. If this is not the case, the signing procedure should be aborted, thus offering protection against this particular fault attack. Care should then be taken to ensure that the verification is not itself vulnerable to fault injection attacks. For the hedged mode itself, another alternative would be to randomly initialise the state. The presented fault attack would then be insufficient to recover the value of ρ'. This countermeasure is not applicable to the deterministic mode, as it introduces non-determinism. Additionally it should be noted that the specification of SHAKE256 makes no claims about the properties of the construction with a randomly initialised state.

More robust countermeasures would require changes in the signing procedure. One such approach may be to move the computation of ρ' into the rejection sampling loop. In that case, an attacker would either be required to predict or affect (through e.g. additional fault injection) the number of times that the rejection sampling is run to inject a single fault during the computation of a signature that is not rejected, or have to inject faults into multiple iterations. This would increase the complexity of an attack. Given the generally high probability of success of the fault injection in this attack and the choice of parameters in Dilithium that inherently keep the number of iterations in the rejection sampling low (see [2]), it is unclear if this approach would be sufficient to prevent an attack.

A different possibility is to increase the complexity of recovering the secret key after a successful fault injection during the computation of ρ'. Here it may be possible to make the value of κ used during the sampling of $\mathbf{y}$ unpredictable (e.g. by increasing its size and initialising it randomly), which would require additional randomness or extension of existing random values. Alternatively, it may be possible to include the attacker-unknown value K (or in hedged mode, rnd, or a combination of both) in the sampling of $\mathbf{y}$, as proposed by [7]. This eliminates the single point-of-failure around the computation of ρ' at the cost of increasing the input size to the hash function during the sampling of $\mathbf{y}$.

X. Conclusion

We presented a practical fault injection attack on a hedged implementation of ML-DSA. We identified settings that consistently skip the desired function without crashing the devices or disrupting other steps of the signature generation. The attack can be applied to other parameter sets and the deterministic mode in ML-DSA, and could even be extended to other algorithms that use SHAKE256 for derivation of secret information.

Our work demonstrates that it is possible to recover the secret key vector in a single attempt with high probability, with

the generated signatures passing verification. This highlights the importance of protecting the calculations of the private random seed ρ', especially when using the hedged mode. Previous work on fault attacks against Dilithium has focused exclusively on the pre-standardisation variant CRYSTALS-Dilithium, while the changes introduced by the ML-DSA variant currently being standardised have not been assessed.

Future work includes developing stronger countermeasures against fault attacks on implementations of PQC algorithms.

Acknowledgement

We would like to thank Erik Thormarker, Håkan Englund, Jakob Sternby, Niklas Lindskog, Kalle Ngo and Ruize Wang for their comments and support.

This work was partially supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation and by the Swedish Civil Contingencies Agency (Grant No. 2020-11632).

References

[1] National Institute of Standards and Technology, "Module-Lattice-Based Digital Signature Standard," National Institute of Standards and Technology, Gaithersburg, MD, Tech. Rep. NIST FIPS 204 ipd, Aug. 2023.

[2] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Dilithium: A lattice-based digital signature scheme," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2018, no. 1, pp. 238–268, 2018.

[3] P. Ravi, M. P. Jhanwar, J. Howe, A. Chattopadhyay, and S. Bhasin, "Exploiting determinism in lattice-based signatures: Practical fault attacks on pqm4 implementations of NIST candidates," in *Proceedings of the 2019 ACM Asia Conference on Computer and Communications Security, AsiaCCS 2019, Auckland, New Zealand, July 09-12, 2019*, S. D. Galbraith, G. Russello, W. Susilo, D. Gollmann, E. Kirda, and Z. Liang, Eds. ACM, 2019, pp. 427–440.

[4] E. Krahmer, P. Pessl, G. Land, and T. Güneysu, "Correction fault attacks on randomized CRYSTALS-Dilithium," Cryptology ePrint Archive, Paper 2024/138, 2024. [Online]. Available: https://eprint.iacr.org/2024/138

[5] P. Ravi, B. Yang, S. Bhasin, F. Zhang, and A. Chattopadhyay, "Fiddling the twiddle constants - fault injection analysis of the Number Theoretic Transform," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2023, no. 2, pp. 447–481, 2023.

[6] V. Q. Ulitzsch, S. Marzougui, A. Bagia, M. Tibouchi, and J. Seifert, "Loop aborts strike back: Defeating fault countermeasures in lattice signatures with ILP," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2023, no. 4, pp. 367–392, 2023.

[7] L. G. Bruinderink and P. Pessl, "Differential fault attacks on deterministic lattice signatures," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2018, no. 3, pp. 21–43, 2018.

[8] National Security Agency, "Announcing the Commercial National Security Algorithm Suite 2.0," 9 2022. [Online]. Available: https://media.defense.gov/2022/Sep/07/2003071834/-1/-1/0/CSA_CNSA_2.0_ALGORITHMS_.PDF

[9] J. P. Mattsson, E. Thormarker, and B. Smeets, "Migration to quantum-resistant algorithms in mobile networks." [Online]. Available: https://www.ericsson.com/en/blog/2023/2/quantum-resistant-algorithms-mobile-networks

[10] A. Abdulrahman, V. Hwang, M. J. Kannwischer, and A. Sprenkels, "Faster Kyber and Dilithium on the Cortex-M4," in *Applied Cryptography and Network Security - 20th International Conference, ACNS 2022, Rome, Italy, June 20-23, 2022, Proceedings*, ser. Lecture Notes in Computer Science, G. Ateniese and D. Venturi, Eds., vol. 13269. Springer, 2022, pp. 853–871.

[11] C. O'Flynn, "Fault injection using crowbars on embedded systems," *IACR Cryptol. ePrint Arch.*, p. 810, 2016. [Online]. Available: http://eprint.iacr.org/2016/810

[12] A. Langlois and D. Stehlé, "Worst-case to average-case reductions for module lattices," *Des. Codes Cryptogr.*, vol. 75, no. 3, pp. 565–599, 2015.

[13] E. Kiltz, V. Lyubashevsky, and C. Schaffner, "A concrete treatment of fiat-shamir signatures in the quantum random-oracle model," in *Advances in Cryptology - EUROCRYPT 2018 - 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29 - May 3, 2018 Proceedings, Part III*, ser. Lecture Notes in Computer Science, J. B. Nielsen and V. Rijmen, Eds., vol. 10822. Springer, 2018, pp. 552–586.

[14] V. Lyubashevsky, "Fiat-Shamir with aborts: Applications to lattice and factoring-based signatures," in *Advances in Cryptology - ASIACRYPT 2009, 15th International Conference on the Theory and Application of Cryptology and Information Security, Tokyo, Japan, December 6-10, 2009. Proceedings*, ser. Lecture Notes in Computer Science, M. Matsui, Ed., vol. 5912. Springer, 2009, pp. 598–616.

[15] A. Barenghi, L. Breveglieri, I. Koren, and D. Naccache, "Fault injection attacks on cryptographic devices: Theory, practice, and countermeasures," *Proc. IEEE*, vol. 100, no. 11, pp. 3056–3076, 2012.

[16] A. Tang, S. Sethumadhavan, and S. J. Stolfo, "CLKSCREW: Exposing the perils of security-oblivious energy management," in *26th USENIX Security Symposium, USENIX Security 2017, Vancouver, BC, Canada, August 16-18, 2017*, E. Kirda and T. Ristenpart, Eds. USENIX Association, 2017, pp. 1057–1074. [Online]. Available: https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/tang

[17] Y. Kim, R. Daly, J. S. Kim, C. Fallin, J. Lee, D. Lee, C. Wilkerson, K. Lai, and O. Mutlu, "Flipping bits in memory without accessing them: An experimental study of DRAM disturbance errors," in *ACM/IEEE 41st International Symposium on Computer Architecture, ISCA 2014, Minneapolis, MN, USA, June 14-18, 2014*. IEEE Computer Society, 2014, pp. 361–372.

[18] J. Breier and X. Hou, "How practical are fault injection attacks, really?" *IEEE Access*, vol. 10, pp. 113 122–113 130, 2022.

[19] P. Ravi, D. B. Roy, S. Bhasin, A. Chattopadhyay, and D. Mukhopadhyay, "Number "Not Used" Once - Practical Fault Attack on pqm4 Implementations of NIST Candidates," in *Constructive Side-Channel Analysis and Secure Design - 10th International Workshop, COSADE 2019, Darmstadt, Germany, April 3-5, 2019, Proceedings*, ser. Lecture Notes in Computer Science, I. Polian and M. Stöttinger, Eds., vol. 11421. Springer, 2019, pp. 232–250.

[20] R. Singh, S. Islam, B. Sunar, and P. Schaumont, "Analysis of EM Fault Injection on Bit-sliced Number Theoretic Transform Software in Dilithium," *ACM Transactions on Embedded Computing Systems*, p. 3583757, Mar. 2023.

[21] L. G. Bruinderink and P. Pessl, "Differential fault attacks on deterministic lattice signatures," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2018, no. 3, pp. 21–43, 2018.

[22] A. Barenghi, G. Bertoni, E. Parrinello, and G. Pelosi, "Low voltage fault attacks on the RSA cryptosystem," in *Sixth International Workshop on Fault Diagnosis and Tolerance in Cryptography, FDTC 2009, Lausanne, Switzerland, 6 September 2009*, L. Breveglieri, I. Koren, D. Naccache, E. Oswald, and J. Seifert, Eds. IEEE Computer Society, 2009, pp. 23–31.

[23] A. Barenghi, G. Bertoni, L. Breveglieri, M. Pellicioli, and G. Pelosi, "Low voltage fault attacks to AES and RSA on general purpose processors," *IACR Cryptol. ePrint Arch.*, p. 130, 2010. [Online]. Available: http://eprint.iacr.org/2010/130

[24] C. Bozzato, R. Focardi, and F. Palmarini, "Shaping the glitch: Optimizing voltage fault injection attacks," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2019, no. 2, pp. 199–224, 2019.

[25] N. Bindel, J. Buchmann, and J. Krämer, "Lattice-based signature schemes and their sensitivity to fault attacks," in *2016 Workshop on Fault Diagnosis and Tolerance in Cryptography, FDTC 2016, Santa Barbara, CA, USA, August 16, 2016*. IEEE Computer Society, 2016, pp. 63–77.

[26] T. Espitau, P. Fouque, B. Gérard, and M. Tibouchi, "Loop-abort faults on lattice-based Fiat-Shamir and hash-and-sign signatures," in *Selected Areas in Cryptography - SAC 2016 - 23rd International Conference, St. John's, NL, Canada, August 10-12, 2016, Revised Selected Papers*, ser. Lecture Notes in Computer Science, R. Avanzi and H. M. Heys, Eds., vol. 10532. Springer, 2016, pp. 140–158.

[27] S. Bauer and F. De Santis, "Forging Dilithium and Falcon signatures by single fault injection," in *Workshop on Fault Detection and Tolerance in Cryptography, FDTC 2023, Prague, Czech Republic, September 10, 2023*. IEEE, 2023, pp. 81–88.

[28] R. Wang, K. Ngo, J. Gärtner, and E. Dubrova, "Single-trace side-channel attacks on CRYSTALS-Dilithium: Myth or reality?" Cryptology ePrint Archive, Paper 2023/1931, 2023. [Online]. Available: https://eprint.iacr.org/2023/1931

[29] Z. Qiao, Y. Liu, Y. Zhou, Y. Zhao, and S. Chen, "Single trace is all it takes: Efficient side-channel attack on Dilithium," Cryptology ePrint Archive, Paper 2024/512, 2024. [Online]. Available: https://eprint.iacr.org/2024/512

[30] M. J. Kannwischer, P. Pessl, and R. Primas, "Single-trace attacks on Keccak," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2020, no. 3, pp. 243–268, 2020.

[31] S. You and M. G. Kuhn, "A template attack to reconstruct the input of SHA-3 on an 8-bit device," in *Constructive Side-Channel Analysis and Secure Design - 11th International Workshop, COSADE 2020, Lugano, Switzerland, April 1-3, 2020, Revised Selected Papers*, ser. Lecture Notes in Computer Science, G. M. Bertoni and F. Regazzoni, Eds., vol. 12244. Springer, 2020, pp. 25–42. [Online]. Available: https://doi.org/10.1007/978-3-030-68773-1_2

[32] ——, "Single-trace fragment template attack on a 32-bit implementation of Keccak," in *Smart Card Research and Advanced Applications - 20th International Conference, CARDIS 2021, Lübeck, Germany, November 11-12, 2021, Revised Selected Papers*, ser. Lecture Notes in Computer Science, V. Grosso and T. Pöppelmann, Eds., vol. 13173. Springer, 2021, pp. 3–23. [Online]. Available: https://doi.org/10.1007/978-3-030-97348-3_1

[33] Q. Do, B. Martini, and K. R. Choo, "The role of the adversary model in applied security research," *Comput. Secur.*, vol. 81, pp. 156–181, 2019.

[34] National Institute of Standards and Technology, "SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions," National Institute of Standards and Technology, Gaithersburg, MD, Tech. Rep. NIST FIPS 202, Aug. 2015.

[35] B. Guido, D. Joan, P. Michaël, and V. Gilles, "Cryptographic sponge functions," 2011.